Machine Learning-Based Motion Intent Recognition for Assistive Robotics

Michal Ridner
Department of Electrical and Computer Engineering
University of Toronto
michal.ridner@mail.utoronto.ca

Onur Ozkaya
Department of Mechanical and Industrial Engineering
University of Toronto
onur.ozkaya@mail.utoronto.ca

Romina Bahrami
Department of Electrical and Computer Engineering
University of Toronto
romina.bahrami@mail.utoronto.ca

Abstract

This project investigates motion intent recognition using wearable sensor data from the UCI Human Activity Recognition dataset. We compare linear and nonlinear models, including a custom multinomial logistic regression and an MLP. Results show that linear models achieve strong performance (about 94–95% accuracy), comparable to more complex methods while requiring less computation. Dimensionality reduction further improves efficiency with minimal performance loss. Overall, the study demonstrates that well-engineered features enable simple models to perform effectively in activity recognition tasks.

Attestation of Teamwork

Michal Ridner: Model Exploration and Implementation + Feature Ablation

Onur Ozkaya: EDA + Model Exploration and Implementation

Romina Bahrami: Model Exploration, Optimization, and Explainability

1 Introduction

Assistive robotic systems, such as exoskeletons, require accurate recognition of human motion intent to provide appropriate support. Traditional rule-based methods rely on manually designed thresholds and often fail to generalize across users and environments.

Machine learning methods offer a data-driven alternative by learning patterns directly from sensor data. In this project, we investigate how machine learning models can classify human activities using wearable sensor measurements, aiming to improve robustness and generalization in assistive robotics applications.

2 Preliminaries and Problem Formulation

We consider a supervised learning problem where the goal is to classify human activities from sensor data. The dataset, the Human Activity Recognition Using Smartphones Dataset from the UC Irvine ML Repository [1], consists of precomputed features extracted from accelerometer and gyroscope signals collected from smartphones worn on the waist by 30 volunteers.

Each sample is represented as a 561-dimensional feature vector, and the target variable corresponds to one of six activities: walking, walking upstairs, walking downstairs, sitting, standing, and laying.

The objective is to learn a function $f : \mathbb{R}^{561} \to \{1, \ldots, 6\}$ that maps input features to activity labels. The model is trained on a subject-wise split, meaning it must generalize to unseen individuals.

3 Design

The overall system consists of the following components:

- **Data Preprocessing:** Loading dataset, label encoding, and feature standardization using z-score normalization.
- **Dimensionality Reduction (Optional):** Principal Component Analysis (PCA) to reduce feature dimensionality while preserving variance.
- **Modelling:** Train the implemented logistic regression and MLP with hyperparameter tuning and regularization on the train split, along with the baseline models (logistic regression and decision tree from sklearn) on the same data split.
- **Evaluation:** Test the trained models on the held-out test set and evaluate their performances. The primary evaluation metrics are accuracy, macro precision, macro recall, and macro F1-score. For a class k, precision and recall are defined as $P_k = \frac{TP_k}{TP_k + FP_k}$ and $R_k = \frac{TP_k}{TP_k + FN_k}$, with $F1_k = \frac{2P_k R_k}{P_k + R_k}$. Macro-averaged metrics are computed by taking the unweighted mean across classes, e.g., Macro-$P = \frac{1}{K}\sum_{k=1}^{K} P_k$ (similarly for recall and F1).

4 Methodology

Different stages of the project, along with the methodology details, are explained below. The implementation is available in the project repository [2].

4.1 Logistic Regression

The primary model is multinomial logistic regression, implemented directly in NumPy. Given input $X \in \mathbb{R}^{N \times 561}$, weights $W \in \mathbb{R}^{6 \times 561}$, and bias $b \in \mathbb{R}^6$, the model computes class probabilities using the softmax function P and its objective is to minimize the average cross-entropy loss L as shown below.

$$P(y = k \mid x_i) = \frac{e^{w_k^T x_i + b_k}}{\sum_{j=1}^{6} e^{w_j^T x_i + b_j}} \qquad L = -\frac{1}{N}\sum_{i=1}^{N}\sum_{k=1}^{6} y_{ik} \log P(y = k \mid x_i)$$

Training is performed using full-batch gradient descent. Hyperparameters such as learning rate and number of iterations are optimized using Optuna.

4.2 Multi-Layer Perceptron (MLP)

As a nonlinear model, we implemented an MLP in PyTorch with the following architectures:

- Small: 561 → 128 → 64 → 6
- Medium: 561 → 256 → 128 → 64 → 6
- Large: 561 → 512 → 256 → 128 → 64 → 6

Each hidden layer uses ReLU activation, batch normalization, and dropout. Training uses mini-batches, the Adam optimizer (weight decay 5×10^{-4}), and cross-entropy loss with label smoothing.

4.3 Baseline Models

We include two baseline models for comparison:

Decision Tree: Implemented using sklearn's `DecisionTreeClassifier` with Gini impurity.

Logistic Regression: Implemented using sklearn's `LogisticRegression` with the `sag` solver.

5 Numerical Experiments

5.1 Data Preprocessing and Analysis

After analyzing the dataset and label distribution (Figure 1), we used the provided train/test split, which partitions subjects (70% training, 30% testing) to evaluate generalization.

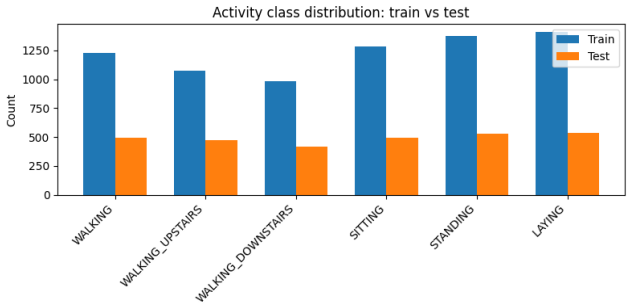


Figure 1: Label distribution of train and test sets

The dataset contains 7,352 training samples and 2,947 test samples, each with 561 features. Feature values were normalized by the dataset authors; we additionally applied z-score standardization (fit on training data only).

5.2 Dimensionality Reduction

We applied PCA to reduce feature dimensionality. Experiments show that over 90% of variance can be retained with fewer than 100 components (Figure 2).

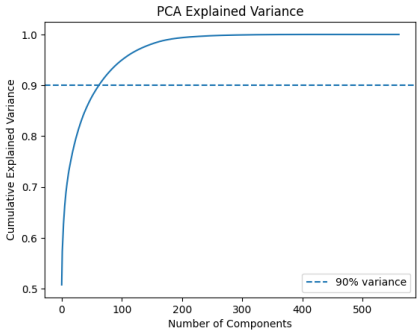


Figure 2: PCA explained variance

To choose the number of components, we searched over candidate principal component counts using a held-out validation split from the train set, stratified by class. Our probe logistic regression is then fit on PCA-transformed training folds; the procedure balances validation accuracy with a stopping rule tied to cumulative explained variance (target 90%). The final PCA is refit on all training samples before transforming both the train and test sets, thereby avoiding test leakage. We then used these reduced features with all of our models.

5.3 Training Procedure

Our logistic regression model was trained using full-batch gradient descent, as this was computationally feasible within the scope of our work. We used the Optuna library for hyperparameter tuning to optimize the number of iterations and the learning rate. To prevent overfitting and maintain efficiency, we incorporated regularization and penalized configurations with excessively large iteration counts or very small learning rates. Hyperparameter tuning was conducted over 30 trials using a 20% stratified subset of the training data. The best-performing parameters were then used to retrain the model on the full training set. Additional trials were also explored, but they did not yield significant improvements.

For the MLP, the original training matrix is split 80% / 20% into train and validation subsets by stratified sampling. A grid search over architecture size, learning rate, and batch size with short runs (15 epochs) picks hyperparameters according to the validation accuracy. The chosen configuration is then trained again for 30 epochs with the same train/validation split before evaluation on the held-out test set.

Sklearn's logistic regression was trained and tuned similarly to our custom implementation. For the decision tree, we used default settings without constraints such as maximum depth or number of nodes, allowing it to fully fit the training data. The model uses Gini impurity to measure class heterogeneity, selecting splits that minimize it and produce purer nodes. As a result, the decision tree can be viewed as a rule-based partitioning of the feature space.

The entire hyperparameter tuning, training, and testing, on full-dimension features, take less than 10 minutes for all models, on an M1 MacBook with 16 GB RAM.

To ensure robustness, models are trained multiple times with different random seeds, and mean performance with standard deviation is reported.

5.4 Model Comparison

All models are evaluated on the held-out test set using accuracy, macro precision, macro recall, and macro F1-score. Macro averaging ensures equal importance across classes. We compared the performance of our implemented models with the baselines using both features with full and reduced dimensions. The results can be found in Table 1. Please note that these results are more recent than those reported in the presentation.

Table 1: Model performance comparison on test set (mean $\pm$ std)

Model	**Accuracy**	**Macro Precision**	**Macro Recall**	**Macro F1**
Decision tree (all features)	0.8590 ± 0.0042	0.8584 ± 0.0045	0.8556 ± 0.0044	0.8559 ± 0.0043
sklearn LR (all features)	0.9520 ± 0.001837	0.9555 ± 0.001512	0.9509 ± 0.001811	0.9521 ± 0.001770
Custom LR (all features)	0.9441 ± 0.000736	0.9474 ± 0.001071	0.9431 ± 0.000764	0.9443 ± 0.000784
MLP (all features)	0.9572 ± 0.0067	0.9600 ± 0.0052	0.9564 ± 0.0071	0.9573 ± 0.0068
Decision tree (reduced features)	0.7615 ± 0.0027	0.7629 ± 0.0031	0.7571 ± 0.0027	0.7587 ± 0.0028
sklearn LR (reduced features)	0.9247 ± 0.001336	0.9254 ± 0.001386	0.9230 ± 0.001222	0.9238 ± 0.001267
Custom LR (reduced features)	0.9224 ± 0.001387	0.9233 ± 0.001233	0.9203 ± 0.001461	0.9212 ± 0.001406
MLP (reduced features)	0.9227 ± 0.0062	0.9240 ± 0.0054	0.9204 ± 0.0069	0.9215 ± 0.0067

5.5 Feature Ablation

Feature ablation was conducted to identify the most influential features for classification. This was performed in two ways: feature subset analysis and weight-based analysis.

5.5.1 Feature Subsets

The dataset's 561 features were grouped into four categories: sensor (gyroscope/accelerometer), component (body/gravity), domain (time/frequency), and axis (X/Y/Z/multi-axis). For each category, the logistic regression model was trained and evaluated using only the corresponding subset of features. Performance was compared using test accuracy, F1-score, and confusion matrices.

5.5.2 Weight Analysis

We further analyzed the learned logistic regression weights to identify the most impactful features. Table 2 presents the most important features per-class, while Table 3 reports the learned bias values for each class.

Table 2: Top two most impactful positive features per activity

	Upstairs	Downstairs	Sitting	Standing	Laying
Feature 1	tGravityAcc-entropy()-X	tBodyAccJerk-max()-X	tGravityAcc-mean()-Y	angle(Y,gravityMean)	angle(X,gravityMean)
Feature 2	tGravityAcc-sma()	fBodyAccMag-energy()	tBodyGyroJerk-arCoeff()-X,1	tBodyGyroJerk-entropy()-X	tGravityAcc-energy()-Y

Table 3: Learned bias values for each activity class

	Walking	Upstairs	Downstairs	Sitting	Standing	Laying
Bias	0.0535	-0.0232	-0.0306	0.0052	-0.0115	0.0066

6 Discussion

On the UCI HAR benchmark with unseen subjects, strong linear models (sklearn and our custom logistic regression) achieve high performance, reaching about 94–95% test accuracy and macro F1 around 0.94–0.95. Our implementation closely matches sklearn, validating both correctness and preprocessing. The sklearn model uses the `sag` solver, which includes variance reduction and likely explains its slight performance advantage. The MLP achieves similar accuracy but offers limited improvement despite higher complexity.

From a computational perspective, our vectorized logistic regression processes all samples simultaneously, enabling efficient training and inference comparable to sklearn baselines. The MLP is also efficient due to its small architecture and PyTorch's optimized backend.

The strong performance of linear models, even after PCA, suggests that the engineered features are largely linearly separable. PCA reduces dimensionality by about $8\times$ while retaining $\sim$90% variance, and logistic regression remains robust with only a small accuracy drop. In contrast, the decision tree performs significantly worse, especially after PCA, likely due to reliance on axis-aligned splits. The MLP also shows some degradation, indicating that PCA better preserves linear relationships than nonlinear ones.

Feature ablation and weight analysis provide further insight into model behavior. Dynamic activities are primarily characterized by rhythm and coordination features, while static activities depend more on posture and gravity-related signals. Time-domain features carry most of the discriminative information, accelerometer data is generally more informative than gyroscope data, and combining feature groups improves performance. Small bias values indicate no strong class preference. Across all models, sitting and standing remain the most difficult activities to distinguish.

Overall, the results indicate that the model has learned meaningful physical patterns from the data.

7 Conclusions

We developed and evaluated machine learning models for motion intent recognition using wearable sensor data. Linear models, particularly logistic regression, achieved strong performance, with our implementation closely matching the sklearn baseline.

Dimensionality reduction using PCA provides an effective trade-off between efficiency and accuracy for linear models, but negatively impacts models that rely more on nonlinear or axis-aligned structures.

Overall, our logistic regression model is both effective and interpretable, demonstrating that well-engineered features combined with simple models can achieve strong performance in activity recognition tasks.

Future work includes exploring second-order optimization methods (e.g., Newton-Cholesky), applying deep learning directly to raw signals, and improving generalization through domain adaptation or subject-wise validation.

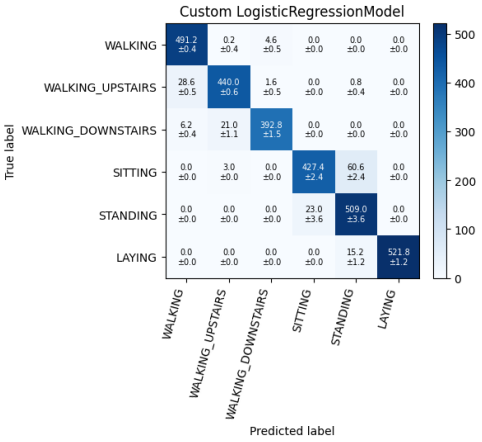
References

[1] Davide Anguita, Alessandro Ghio, Luca Oneto, Xavier Parra, and Jorge L. Reyes-Ortiz. A public domain dataset for human activity recognition using smartphones. In *Proceedings of the 21st European Symposium on Artificial Neural Networks, Computational Intelligence and Machine Learning (ESANN)*, Bruges, Belgium, April 2013. 24–26 April 2013.

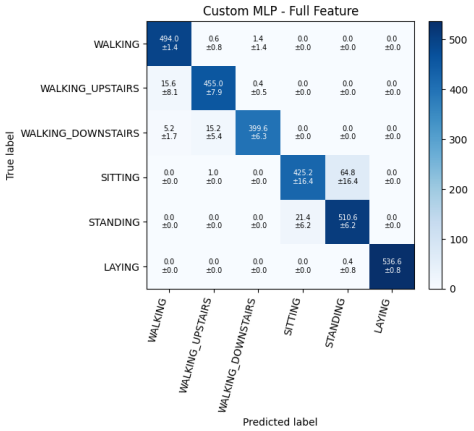
[2] Onur Ozkaya Romina Bahrami, Michal Ridner. Machine learning-based motion intent recognition for assistive robotics. `https://github.com/rominaba/motion-intent-recognition-for-assistive-robotics`, 2026.

Appendix

7.1 Appendix A: Confusion Matrices

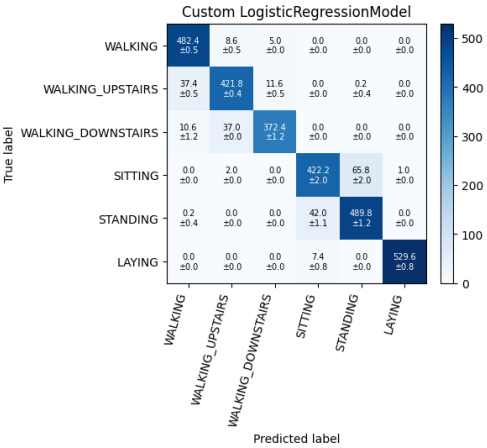


(a) Confusion matrix for custom logistic regression using full features.

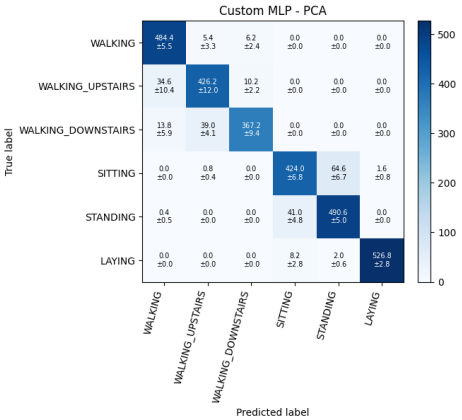


(b) Confusion matrix for MLP using full features.

Figure 1: Confusion matrices for both models using full feature sets.



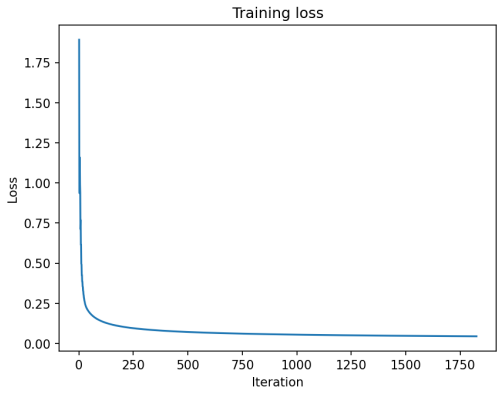
(a) Confusion matrix for custom logistic regression after PCA.



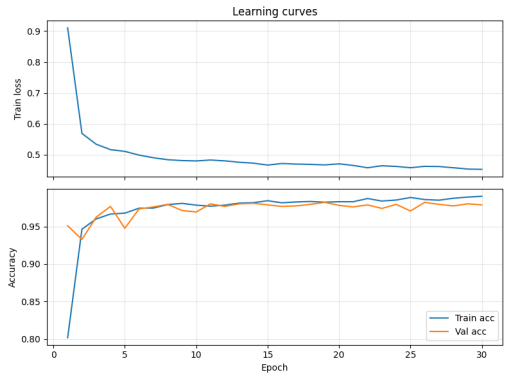
(b) Confusion matrix for MLP after PCA.

Figure 2: Confusion matrices for both models after PCA.

7.2 Appendix B: Learning Curves

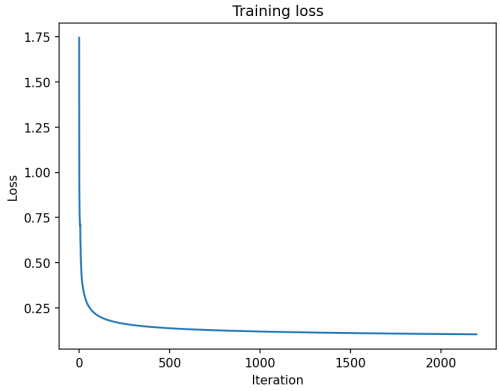


(a) Learning curve for custom logistic regression using full features.

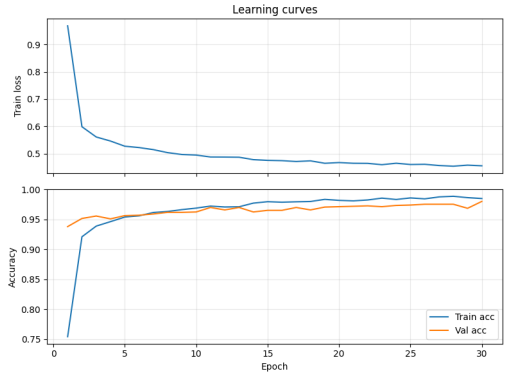


(b) Learning curve for MLP using full features.

Figure 1: Learning curves for both models using full feature sets.



(a) Learning curve for custom logistic regression after PCA.



(b) Learning curve for MLP after PCA.

Figure 2: Learning curves for both models after PCA.

AI Usage

AI tools were used in a limited capacity to support research, deepen understanding of certain techniques, and improve the clarity of documentation and code. All AI-generated suggestions were critically evaluated by the team before being utilized.

Contest for Information Sharing

Consent for Sharing Project Materials

- All group members consent to allow the project materials (report, slides, and presentation recording) to be shared on the course page for future students.

Consent for Use of Project Information in Statistical Analysis

- All group members consent to allow anonymized information about the project (e.g., topic, methods, outcomes, grades) to be used by the instructor for statistical analysis and course improvement.

Group Identification

- Group number: 27
- Names of group members: Romina Bahrami, Onur Ozkaya, Michal Ridner
- Signature of of group members: *Romina Bahrami, Onur Ozkaya, Michal Ridner*
- Date: April 12, 2026